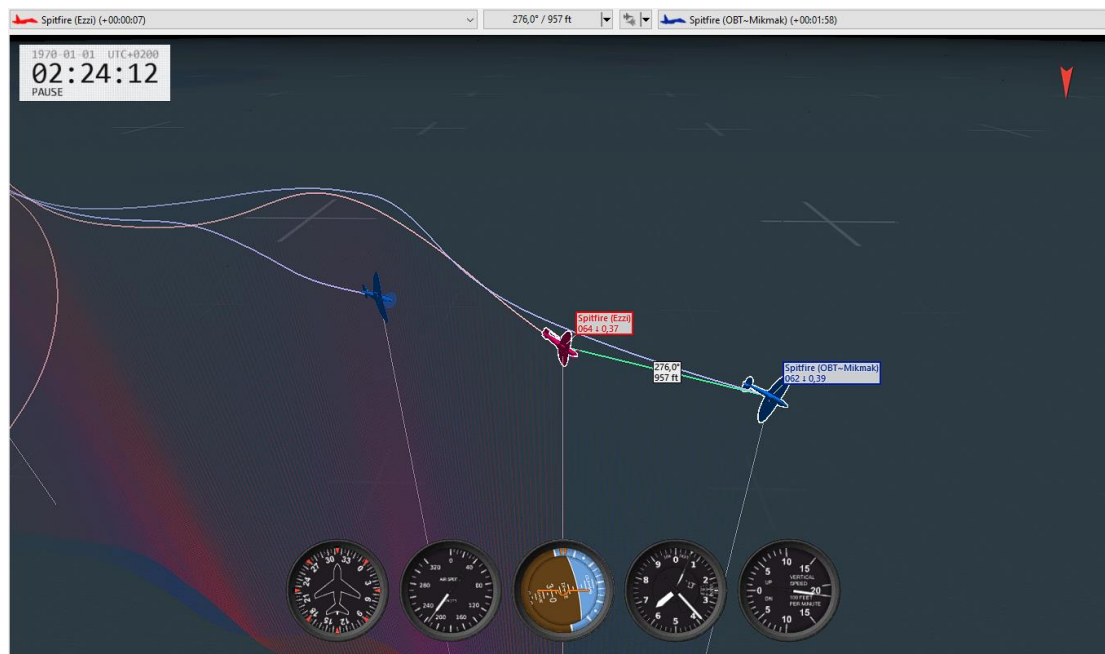


Tacview Recorder V2.0

pour IL2-Sturmovik : Cliffs of Dover Blitz & Tobruk

Par FlyBy (anciennement OBT~Eazy).



Premier débriefing au monde sur Tacview en utilisant les données de vol de COD (vol d'entraînement OBT).

<https://obteam.forumsactifs.net>

Fonctionnalités de la V2.0

- **Plus de code à intégrer dans votre mission.**
- [Nouvelle classe de base](#) pour mission
- Les unités terrestres, navales et statiques ([Acteur](#)) (autres que joueurs et avions) sont affichées dans [Tacview](#).
- Nouvelle méthode de correspondance des modèles 3D de CloD et Tacview.
- Les dommages, la mort ou destruction d'un acteur gérés et affichés dans Tacview.
- Les acteurs endommagés ou détruits changent de couleur.
- 3 modes disponibles (dogfight, normal, big mission)
- Événements impliquant des joueurs injectés dans le journal des événements de Tacview.
- Affichage des explosions de bombes dans Tacview.
- Création et suppression de points de repère à la demande.
- Réduction de la taille du fichier Tacview (compression zip).
- Fonctionne dans son propre [thread](#).
- Amélioration des timers (une mission simple peut être accélérée jusqu'à 16x).
- Prêt pour les futurs DLC ([correspondance des modèles 3D](#)).
- Données de vol compatibles avec le format Tacview (acmi 2.1).
- Faible impact sur les performances.
- Démarrage, pause, arrêt, démarrage retardé de l'enregistrement des données programmable.
- Les données de chaque joueur et des IA sont enregistrées dans un seul fichier.

Limitations :

- Comme pour la V1, la balistique des projectiles (balles, obus, bombes) n'est pas disponible.
- Les fichiers de données de vol sont enregistrés sur le serveur exécutant la mission.

Avant-propos.

En tant que développeur de cet add-on, je ne suis impliqué ni directement ni indirectement dans le développement, la maintenance ou le support technique du logiciel Tacview, et je ne fais pas partie de la société Raia Software sous quelque forme que ce soit.

Mon opinion personnelle est que le logiciel Tacview est très utile pour le débriefing de nos vols en plus d'offrir un rapport qualité/prix élevé. De plus, le format de fichier utilisé par Tacview est public, gratuit et d'une structure simple.

Ces raisons ont motivé l'implication de mon temps libre dans le développement de cet add-on. J'espère qu'il rencontrera toutes vos attentes.

Cette nouvelle version a été l'occasion de recréer le [Tacview Recorder](#) à partir de zéro (pas de compatibilité descendante avec la version précédente).

Le but principal de la réécriture était de permettre l'utilisation du [Recorder](#) avec le moins de code à intégrer de la part du concepteur de mission. Pour ce faire, une nouvelle classe de mission, [TacviewMission](#), a été développée. Cette classe dérive de la classe standard `AMission`. En dérivant ses missions de la nouvelle classe, le concepteur n'a plus aucune ligne de code à ajouter pour activer le [recorder](#) car les événements nécessaires sont interceptés et traités automatiquement par la nouvelle classe.

Il fallait aussi que les [acteurs](#) terrestres autres que les avions comme les navires, trains, tanks, flak,... mobiles ou non soient traités et visibles dans Tacview, de même que les dommages et les explosions de bombes.

Le recorder devait être utilisé indifféremment sur n'importe quel type de mission aussi bien mission simple, sur serveur privé (salon) ou sur serveur dédié.

Un soin particulier a été apporté aux performances. Le recorder tourne dans son propre [thread](#) (flux d'exécution), le temps d'interception des événements et de leur traitement sont réduits au minimum (voir le chapitre « [Comment ça marche](#) »).

Clause de non-responsabilité.

Le présent add-on est fourni tel quel, sans aucune garantie expresse ou implicite quant à sa qualité, sa performance ou son adéquation à un usage particulier. L'utilisateur assume tous les risques liés à son utilisation. L'auteur de l'add-on et les personnes associées à son développement ne seront en aucun cas responsables des dommages directs, indirects, accessoires, spéciaux ou consécutifs découlant de l'utilisation ou de l'impossibilité d'utiliser ce logiciel, même s'ils ont été informés de la possibilité de tels dommages.

L'utilisateur est responsable de l'utilisation de l'add-on et doit se conformer à toutes les lois et réglementations en vigueur. Il est recommandé à l'utilisateur de sauvegarder régulièrement ses données pour éviter toute perte en cas de dysfonctionnement du logiciel.

L'utilisation de cet add-on implique l'acceptation sans réserve des termes de cette clause de non-responsabilité.

Rendons hommage avec respect et gratitude à tous les pilotes, bombardiers, mitrailleurs, mécanos, ainsi que tout homme et femme, qui, avec courage et dévouement inébranlables, ont offert le sacrifice ultime pour leur patrie, défendant avec bravoure les cieux de leur nation, quel que soit leur camp.

Remerciements

- Merci à OBT~Mikmak, OBT~Lionel et OBT~Brunov pour leur participation aux tests du recorder en multi-joueurs.
- Merci à ATAG_Lenny pour ses tests du recorder en mode mission simple.
- Merci à Artist et Scott pour leur aide apportée aux tentatives de corrections des coordonnées géographiques.
- Merci aux bénévoles passés, présents, et à venir, membres de l'équipe de Team Fusion Simulations de maintenir et améliorer ce simulateur sur leur temp libre.
- Merci aux concepteurs de missions mono et multi joueurs qui aident par leur travail et leur rigueur à maintenir l'intérêt des joueurs.
- Et bien sûr Merci aux joueurs de l'ombre ou ace des aces qui, par leur fairplay et leur bonne humeur, me font continuer à voler.

Table des matières.

Lexique.	6
Comment ça marche.	7
Intégration dans vos missions.	9
Nouvelle mission.	9
Mission existante (la mission contient déjà des scripts).	9
Distribuez vos missions.	10
Méthodes de base utilisées par le recorder.	11
Paramétrer et contrôler le recorder.	12
Propriétés.	12
DestinationFolder	12
MissionType	12
ShowPlayer	12
ShowPlayerAsHuman	13
StartDelay	13
ZipFinalFile	13
Méthodes.	14
StartRecorder()	14
PauseRecorder()	14
StopRecorder()	14
DisableRecorder()	14
AddWaypoint(name, x, y, z, army= 0)	14
RemoveWaypoint(name)	14
AddBookmark(message)	15
Représentation des dommages.	16
Evènements dans Tacview.	18
Evènements sauvegardés.	18
Afficher le journal des évènements.	18
Sélectionner un évènement.	19
Correspondances des modèles 3D.	20
Tableau des correspondances (avions seulement).	20
Liens utiles.	22
Problèmes connus.	23

Lexique.

Acteur : tout objet, mobile ou non qui peut interférer avec le jeu (joueur, avion, équipage, navire, tank, train, flak, ...)

Fichier CSV (Comma-Separated Values) : liste organisée de données où chaque ligne représente un enregistrement et chaque colonne représente un attribut ou une caractéristique. C'est un format simple et couramment utilisé pour stocker des données tabulaires, souvent dans des feuilles de calcul ou des bases de données.

Multithreading : le multithreading est largement utilisé dans les applications informatiques pour des tâches telles que le traitement des données en arrière-plan, la manipulation des interfaces utilisateur, l'exécution de plusieurs opérations simultanément, et bien d'autres encore. Il permet d'améliorer la réactivité des applications, d'optimiser l'utilisation des ressources du système et d'accélérer le traitement des tâches parallèles.

Recorder : raccourci pour Tacview Recorder

Tacview : logiciel de visualisation et d'analyse de données de vol développé par RAIA Software Inc. Disponible à <https://www.tacview.net>

Tacview Recorder : add-on développé pour intercepter et enregistrer les données de vol, les objets de mission, les dommages et explosions compatible au format du logiciel Tacview (génère un fichier en format acmi).

TacviewMission : classe de base pour Tacview Recorder. Cette classe intercepte les événements nécessaires au recorder. Elle dérive de la classe AMission.

Tampon : une mémoire tampon est une zone temporaire où l'ordinateur stocke des données en attente d'être utilisées. C'est une zone de transit où les données passent avant d'être traitées ou déplacées vers leur destination finale.

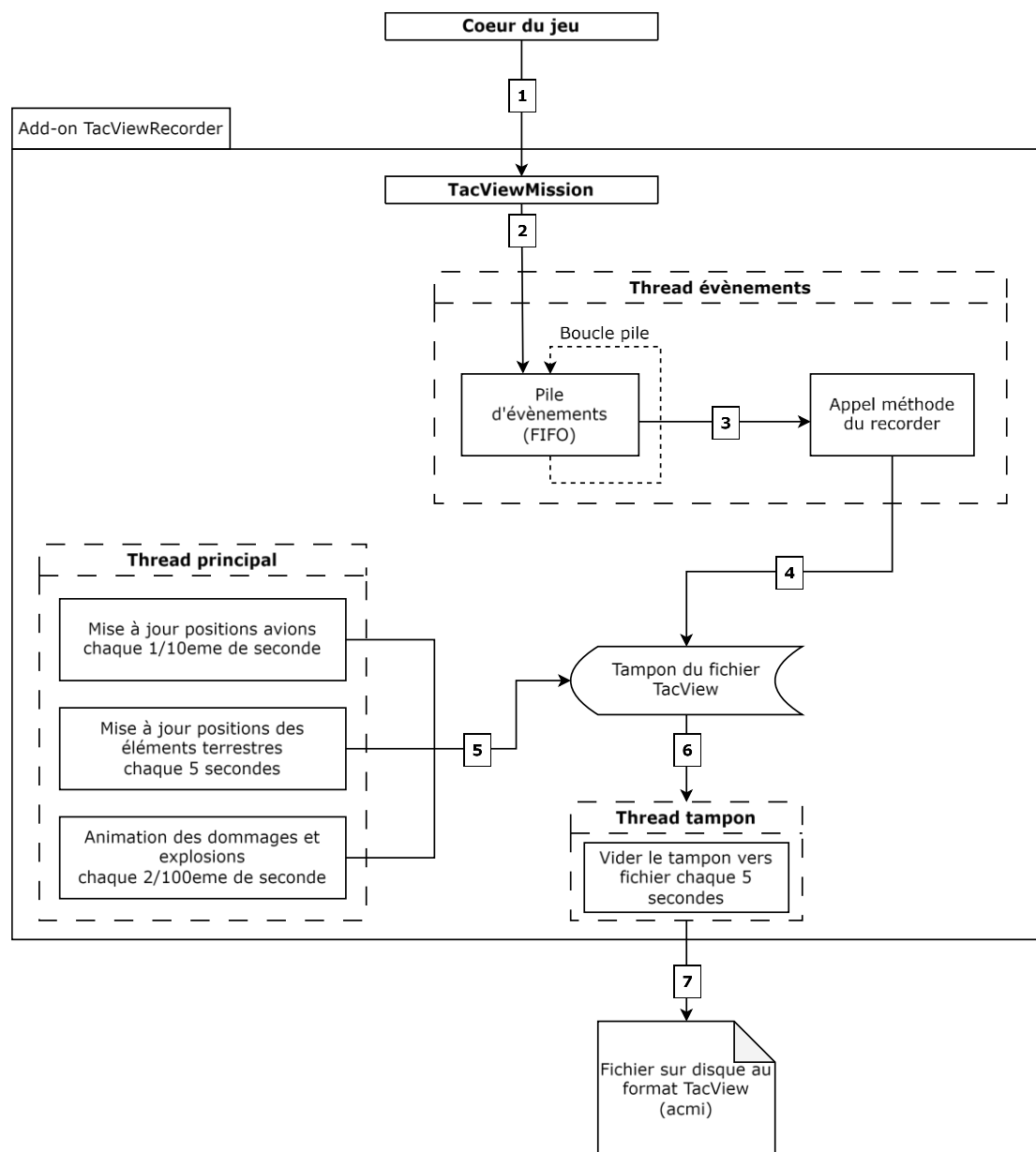
Thread (flux d'exécution) : en termes informatiques, un thread est une unité d'exécution au sein d'un programme en cours d'exécution, et un thread est une partie de ce programme qui peut être exécutée indépendamment des autres parties. Le recorder utilise ses propres threads pour traiter les événements déclenchés lors du jeu (création d'acteur, dommage sur acteur, mort, destruction, explosion, ...) et n'interfère pas avec le thread principal du jeu. Ceci permet de garantir des performances très peu altérées.

Watchdog : programme développé par Colander et maintenu par Artist de l'équipe TFS. Ce programme permet de gérer l'arrêt et le démarrage de missions sur un serveur dédié (pas un salon).

Comment ça marche.

Ce chapitre présente sommairement le fonctionnement du [recorder](#) en termes techniques et peut amener quelques confusions chez le lecteur qui n'est pas au fait des techniques de programmation et de fonctionnement d'un programme informatique.

Représentation simplifiée de l'architecture du recorder.



Le recorder est composé de 3 [threads](#) indépendants et d'une mémoire [tampon](#) (limitation des opérations d'écriture sur le disque dur de l'ordinateur). Cette architecture permet de dissocier l'exécution du code du recorder de celui du jeu et donc d'affecter le moins possible les performances de notre simulateur préféré, que ce soit en mission simple ou en multi-joueurs.

Le thread "événements" intercepte les événements issus du jeu. Le terme événement n'est pas correctement utilisé ici car techniquement c'est un appel de fonction mais, pour la plupart, ces appels sont liés à des événements de jeu comme les dommages, les destructions, la connexion d'un joueur, etc., pour plus de clarté c'est le terme qui sera utilisé.

Le thread "principal" est responsable des mises à jour des positions des [acteurs](#) (joueurs, avions, unités terrestres) ainsi que des animations des dommages et des explosions. Pour limiter la taille du fichier Tacview sur disque, les délais des différentes mises à jour sont optimisés en fonction du type d'acteurs. Pour les joueurs et les avions, les mises à jour sont exécutées tous les 1/10ème de seconde tandis que pour les unités terrestres, plus lentes, 5 secondes sont suffisantes. Le traitement des animations des dommages et explosions, si existants, s'exécute tous les 2/100ème de seconde.

Le thread "tampon" écrit le contenu du tampon dans le fichier (sur disque dur) au format Tacview toutes les 5 secondes.

Le nom du fichier est formaté comme suit :

Tacview-AnnéeMoisJour-HeureMinuteSeconde-COD-NomDeLaMission.txt.acmi

Si le fichier est compressé, l'extension est zip.acmi

Le thread principal utilise le temps tiré de `Time.current()` qui correspond au temps réel de la mission, ce qui permet au recorder de fonctionner même si la mission est accélérée, décélérée ou mise en pause.

Les threads sont synchronisés par l'utilisation d'un drapeau (flag) afin d'éviter le blocage des threads (deadlock) non représenté ici.

Remarque : l'ordre des opérations décrites ci-dessous n'est pas représentatif de l'ordre réel d'exécution de ces opérations car le recorder opère dans un environnement [multithreading](#).

- 1** Le cœur du jeu envoie un évènement à la classe TacviewMission.
- 2** Cet évènement est intercepté par le recorder qui le place sur une pile d'attente et rend la main au cœur du jeu immédiatement (le jeu n'a pas à attendre le traitement de cet évènement et ses performances ne sont pas affectées). Cette pile est organisée de telle sorte que le 1^{er} élément entré est le 1^{er} sorti (FIFO) par la boucle parcourant la pile. Ces opérations se déroulent dans le thread "évènements".
- 3** La méthode interne correspondante à l'évènement est exécutée.
- 4** La méthode peut éventuellement envoyer des données dans le tampon selon l'évènement (création de joueur ou d'acteur, messages relatifs à l'évènement, ...).
- 5** Le thread "principal" envoie les positions (avions et unités terrestres) et les "animations" dans le tampon.
- 6** et **7** Le thread "tampon" récupère le contenu du tampon et l'écrit dans le fichier sur disque.

Intégration dans vos missions.

Ce chapitre ne traite pas de la création de mission et de scripts. Nous supposons que vous avez suffisamment de connaissances pour comprendre ce qui suit. Si ce n'est pas le cas, vous trouverez certainement de l'aide sur le forum [ATAG](#) et en particulier dans la section [Full Mission Builder and Scripts](#).

Cette nouvelle version permet d'activer le recorder sans avoir à ajouter de code dans vos missions grâce à la nouvelle classe de base TacviewMission.

Nouvelle mission.

Si le fichier .cs n'existe pas encore pour votre mission, vous pouvez utiliser le modèle (Mission_Template.cs) inclus dans les fichiers installés. Ce modèle est disponible dans le dossier suivant :

C:\Users\NomUtilisateur\Documents\1C SoftClub\il-2 sturmovik cliffs of dover\Tacview\Templates
Copiez le fichier MissionTemplate.cs à l'emplacement de votre mission (où se trouve votre fichier .mis) et renommez-le avec le nom de votre mission.

Par exemple si votre mission se nomme Dogfight.mis, renommez le modèle en Dogfight.cs
Soyez attentif à ce que l'extension soit bien .cs

Il ne vous reste plus qu'à lancer votre mission, le recorder enregistrera les données automatiquement.

Mission existante (la mission contient déjà des scripts).

- Ajouter la référence à TacViewRecorder.dll au début du fichier .cs de la mission.

```
//$reference TacviewRecorder.dll
using TacviewRecorder;
```

Attention : veillez à ne pas supprimer les références déjà existantes dans votre fichier.

- Remplacer la classe de base **AMission** par **TacviewMission**.

```
public class Mission : AMission
```

devient

```
public class Mission : TacviewMission
```

Attention : si vous appelez une des méthodes utilisées par la classe de base TacviewMission (voir [méthodes utilisées](#)), veillez à appeler la méthode de base dans votre code.

Par exemple si vous interceptez la méthode OnPlaceEnter :

```
public override void OnPlaceEnter(Player player, AiActor actor, int placeIndex)
{
    base.OnPlaceEnter(player, actor, placeIndex); // ligne à ajouter

    // VOTRE CODE ICI //
}
```

NB : cela fait partie des bonnes pratiques de programmation de toujours appeler la méthode de la classe de base en cas d'héritage.

Remarque : les éventuelles missions secondaires chargées par la méthode `gpPostMissionLoad` ne doivent pas obligatoirement hériter de la classe `TacviewMission` mais elles peuvent. Les scripts liés à ces missions n'ont pas d'incidence sur le recorder.

En plus des remarques concernant [les méthodes utilisées](#) par le recorder, il faut tenir compte de l'utilisation éventuelle des excellents scripts disponibles sur le forum ATAG.

<https://theairtacticalassaultgroup.com/forum/showthread.php?t=34571>

Vous devez vous assurer que ces scripts appellent les fonctions de la classe de base `TacviewMission`.

Avant d'arrêter la mission par script par appel de la méthode

`Process.GetCurrentProcess().Kill()`, veuillez à appeler la méthode

`Battle.OnBattleStopped()` afin de libérer les ressources utilisées par le recorder et la fermeture correct du fichier Tacview pour éviter que ce fichier soit corrompu.

Il en va de même si vous utilisez le [watchdog](#) (rotation de missions).

<https://theairtacticalassaultgroup.com/forum/showthread.php?t=35224&highlight=watchdog>

https://www.team-fusion-simulations.de/tfsresources/?&path=03_Servers

Distribuer vos missions.

Si vous distribuez vos missions (fichiers `.mis` et `.cs`) à d'autres joueurs, il faut que ces autres joueurs aient installé préalablement le recorder sur leur ordinateur.

Soyez attentif au paramètre `DestinationFolder` si vous l'avez modifié. Il y a de grandes chances que le répertoire de destination des fichiers TacView n'existe pas sur les PC des autres joueurs.

Méthodes de base utilisées par le recorder.

Si vos scripts de mission (fichier .cs) appellent une des fonctions suivantes, il faut absolument appeler la méthode de la classe de base dans votre code pour assurer le bon fonctionnement du recorder, voir le chapitre "[Intégration du recorder](#)".

Par exemple si vous interceptez la méthode OnPlaceEnter :

```
public override void OnPlaceEnter(Player player, AiActor actor, int placeIndex)
{
    base.OnPlaceEnter(player, actor, placeIndex); // ligne à ajouter

    // YOUR CODE HERE BELOW //
}
```

NB : cela fait partie des bonnes pratiques de programmation de toujours appeler la méthode de la classe de base en cas d'héritage.

```
Init
OnActorCreated
OnActorDamaged
OnActorDead
OnActorDestroyed
OnAircraftCrashLanded
OnAircraftCutLimb
OnAircraftDamaged
OnAircraftKilled
OnAircraftLanded
OnAircraftLimbDamaged
OnAircraftTookOff
OnBattleInit
OnBattleStarted
OnBattleStoped
OnBombExplosion
OnBuildingKilled
OnMissionLoaded
OnPersonHealth
OnPersonMoved
OnPersonParachuteFailed
OnPersonParachuteLanded
OnPlaceEnter
OnPlaceLeave
OnPlayerArmy
OnPlayerConnected
OnPlayerDisconnected
OnStationaryKilled
```

Paramétrer et contrôler le recorder.

Propriétés.

Ces propriétés **DOIVENT** être utilisées dans la méthode *public override void Init()* de votre mission, si pas, elle sont ignorées.

DestinationFolder

Choix du dossier de destination du fichier Tacview.

Type : chaîne de caractères

Par défaut, le dossier de destination est :

C:\Users\NomUtilisateur\Documents\1C SoftClub\il-2 sturmovik cliffs of dover\Tacview

Exemple :

```
DestinationFolder = @"D:\Tacview\";
```

MissionType

Définit le type de mission en cours. Selon le type, certains objets sont ignorés.

Type : Enum TypeOfMission

Valeurs possibles :

TypeOfMission.BigMission = Avions + unités terrestres + objets statiques.

TypeOfMission.Dogfight = Avions uniquement.

TypeOfMission.Normal = Avions + unités terrestres.

Par défaut, le type de mission est BigMission

ShowPlayer

Dans Tacview, visualise les joueurs sous la forme d'une pyramide verte.

Type : booléen (true/false)

Par défaut, les joueurs sont visualisés dans Tacview (valeur = true).

Exemple :

```
ShowPlayer = true;
```



(ShowPlayer = true)

Si plusieurs joueurs sont à bord de l'avion, ils sont empilés.



ShowPlayerAsHuman

Dans Tacview, visualise le joueur sous la forme d'un personnage vert.

Type : booléen (**true/false**)

Par défaut, les joueurs sont représentés sous forme de pyramide verte.

A utiliser avec la propriété `ShowPlayer`.

Exemple :

```
ShowPlayerAsHuman = true;
```



`ShowPlayerAsHuman = true` (`ShowHuman` doit être égal à `true`)

StartDelay

Fixe un délai (en seconde) au démarrage du recorder.

Type : Entier (int)

Utile si le concepteur de mission estime qu'il n'y a pas d'action à enregistrer dans Tacview avant un certain temps.

Valeur possible

Valeur = 0 : démarrage immédiat du recorder

Valeur > 0 : démarrage x seconde(s) après le chargement de la mission.

Valeur = -1 : le recorder ne démarre pas automatiquement et doit être démarré à l'aide de la fonction `StartRecorder()`

La temporisation peut être annulée par l'appel de la fonction `StartRecorder()` qui annule le démarrage temporisé et démarre le recorder immédiatement.

La méthode `StopRecorder()` annule le démarrage temporisé et arrête définitivement le recorder.

La méthode `PauseRecorder()` n'a pas d'effet sur le démarrage temporisé.

Par défaut, le recorder démarre automatiquement au chargement de la mission.

Exemple :

```
StartDelay = 300; // Démarre dans 300 secondes
```

ZipFinalFile

Le fichier Tacview est compressé au format ZIP. Permet de réduire jusqu'à 70% la taille du fichier non compressé.

Type : booléen (**true/false**)

Par défaut, le fichier Tacview est compressé.

Exemple :

```
ZipFinalFile = false; // ne pas compresser le fichier
```

Méthodes.

StartRecorder()

Démarre le recorder à la demande après une mise en pause ou un démarrage temporisé.
Voir les méthodes `PauseRecorder()` et `StartDelay()`
Peut être appelée par un trigger ou une entrée de menu.
Par défaut le recorder démarre automatiquement au chargement de la mission.

PauseRecorder()

Suspend le recorder.
L'enregistrement des : positions, dommages, explosions, évènements est suspendu.
Les dommages et destructions sont cependant gardés en mémoire et traités lors du redémarrage du recorder (changement du statut et couleur des acteurs).
Peut être appelée par un trigger ou une entrée de menu.

StopRecorder()

Arrêt définitif du recorder. **Ne peut pas être redémarré.**
A ne pas confondre avec la commande `PauseRecorder()`
Peut être appelée par un trigger ou une entrée de menu.

DisableRecorder()

Interdire le démarrage du recorder. Permet de faire tourner une mission sans tenir compte du recorder et sans à devoir changer la classe de base de la mission.
Tous les appels aux méthodes du recorder sont ignorées.
A appeler dans la méthode : `public override void Initd()` de la mission de base.

AddWaypoint(name, x, y, z, army= 0)

Positionne un point de repère dans Tacview aux coordonnées X, Y et Z identifié par un nom unique.
Les valeurs des coordonnées doivent être égales ou supérieures à 0.
Le paramètre army (0 par défaut) définit la couleur du point de repère (0 = vert, 1 = rouge, 2 = bleu).



RemoveWaypoint(name)

Supprime le point de repère identifié par son nom.

AddBookmark(message)

Injecte un message dans le [journal des événements](#) de Tacview.

Permet au concepteur de mission d'ajouter un point de repère dans la ligne du temps de Tacview.

Attention, pas d'effet si utilisé dans la fonction *public override void Init()* (le recorder n'a pas encore démarré sa ligne du temps).

A utiliser de préférence dans un trigger pour synchroniser correctement le message.

Par exemple :

```
AddBookmark("Bombardiers détectés au large de Douvres.");
```

Représentation des dommages.

Les joueurs et [acteurs](#) blessés, morts, endommagés ou détruits changent de couleur dans [Tacview](#).

Pour les joueurs, les couleurs sont : vert, jaune, blanc



Joueur OK



Joueur blessé



Joueur mort
(l'avion passe en blanc)

Avions et unités terrestres.

Armée bleue : bleu, violet, blanc

Armée rouge : rouge, orange, blanc



Pas de dommage



Endommagé



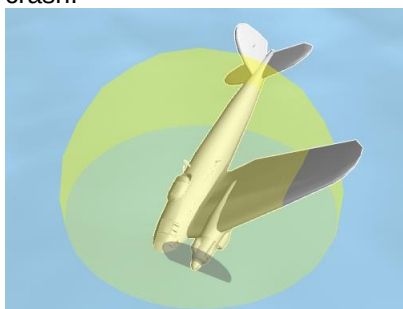
Inopérationnel OU pilote tué.

Remarque : si un joueur reprend la place du pilote tué ET que l'avion est encore opérationnel, l'avion reprend la couleur définie selon son armée et son état (pas de dommage ou endommagé)

Lorsqu'un joueur ou un acteur subit un dommage, il est entouré d'une bulle jaune temporaire.



Un avion détruit prend la couleur blanche, il est entouré d'une bulle jaune temporaire lors du crash.



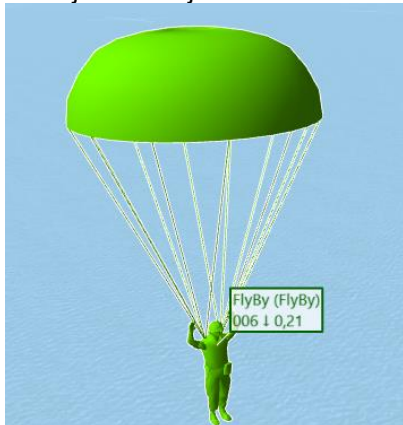
Explosions de bombes.

Les explosions de bombes sont représentées par des bulles jaunes temporaires (5 secondes).

Le diamètre de la bulle est calculé d'après la masse de la bombe.



Si un joueur s'éjecte de son avion, il est représenté par un parachutiste.



Actuellement, l'orientation du parachutiste est incorrecte dans la version 1.9.3 de Tacview
Une alternative anachronique est utilisée à la place.

Evènements dans Tacview.

Ce chapitre n'est pas un mode d'emploi du logiciel Tacview.

Veuillez vous référer à la documentation en ligne ici (en anglais) :

https://tacview.fandom.com/wiki/The_basics

Evènements sauvegardés.

Le recorder n'injecte que les évènements impliquant explicitement le ou les joueurs dans le journal d'évènements de Tacview ceci pour faciliter la recherche d'un événement particulier et d'éviter de surcharger le journal.

Il s'agit, entre-autres, des évènements suivants :

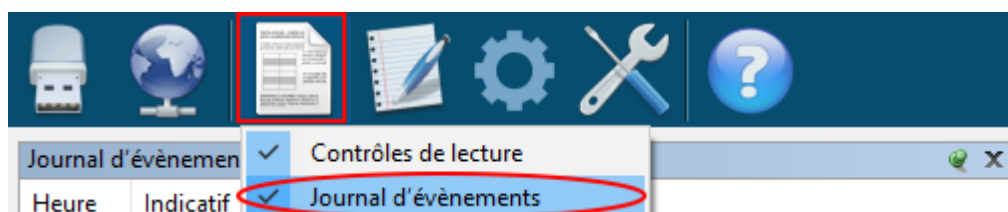
- Le joueur se connecte ou se déconnecte du serveur,
- Le joueur sélectionne une armée,
- Le joueur prend place dans un avion (solo ou multi-équipage),
- Le joueur décolle,
- Le joueur a participé à la destruction d'un acteur (joueur, avion ou unité terrestre),
- Le joueur est blessé,
- Le joueur est tué par un ou plusieurs joueurs ou une IA,
- Le joueur saute en parachute, ...

Remarque : lorsque plusieurs joueurs se trouvent à bord d'un avion multi-équipage, chaque joueur est impliqué dans l'évènement (sauf en cas de blessure) aussi bien en tant qu'initiateur que victime.

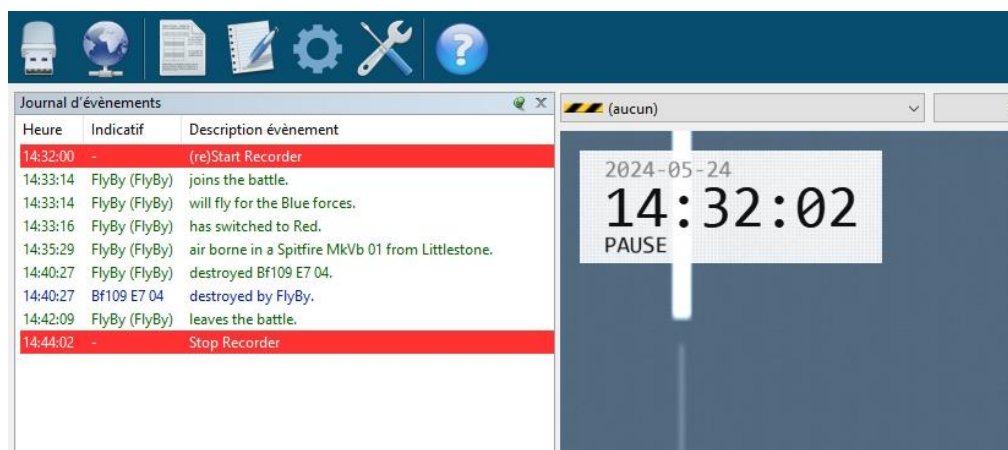
Afficher le journal des évènements.

- Cliquer sur l'icône "Analyse" dans la barre de menu de Tacview
- Sélectionner "Journal d'évènements" dans la liste.

Il n'est pas utile d'ouvrir le journal de vol de Tacview car il contient des évènements générés automatiquement et n'est pas assez précis.

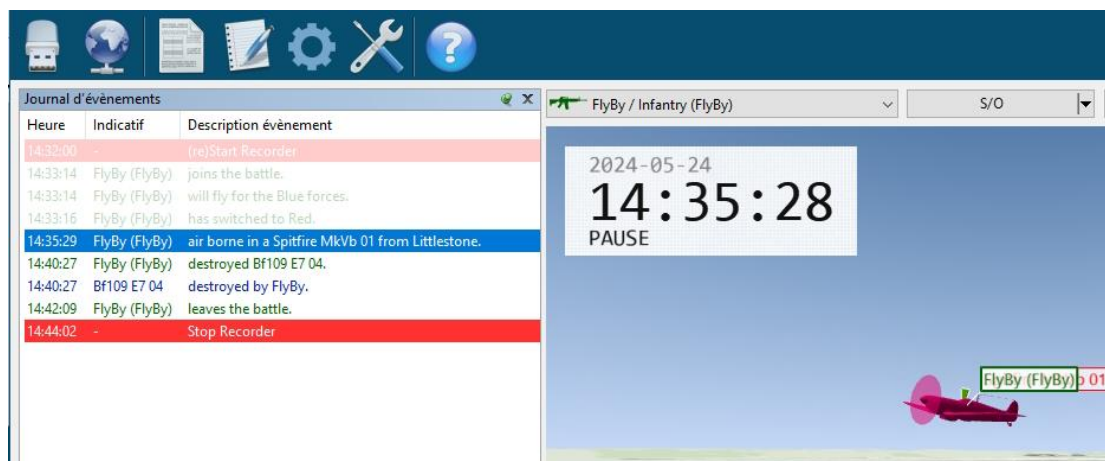


Si un fichier est chargé et contient des évènements, ils seront affichés dans le journal.



Sélectionner un évènement.

Pour se rendre à l'instant de l'évènement dans la ligne du temps de Tacview, cliquer sur ligne. L'acteur initiateur de l'évènement sera automatiquement sélectionné par Tacview, ici, le pilote FlyBy.



Correspondances des modèles 3D.

Ce chapitre explique, à titre informatif seulement, comment le [recorder](#) définit la représentation des types et modèles des acteurs dans [Tacview](#)

La correspondance des modèles 3D entre Cliffs of Dover et Tacview est faite par l'intermédiaire d'un fichier texte au format [CSV](#) généré à partir d'un tableur Excel.

Ce fichier est localisé dans le dossier suivant :

C:\Users\NomUtilisateur\Documents\1C SoftClub\il-2 sturmovik cliffs of dover\Tacview\TacviewParam et se nomme TacviewModelMapping.csv

Cette méthode a été choisie pour éviter une recompilation et une réinstallation complète du recorder si une correspondance doit être modifiée plus tard (ajout d'un DLC par exemple).

Il est déconseillé de modifier le contenu du fichier de correspondance au risque de rendre le recorder inutilisable.

Lors de l'initialisation du recorder le contenu du fichier est traité et organisé dans une liste interne en fonction du type d'[acteur](#) et de son nom interne dans le jeu.

Lors de la création d'un acteur par le jeu, le recorder recherche la correspondance dans sa liste et définit la représentation dans Tacview.

Par exemple, si le jeu crée un avion de type Spitfire IIb de nom interne tobruk:Aircraft.SpitfireMkIIb, l'avion dans Tacview sera représenté par un modèle 3D du nom FixedWing.Spitfire.obj et son identifiant sera "Spitfire MkIIb" suivi de son n° d'ordre de création.

Les modèles 3D disponibles se trouvent sur le site [GitHub de Tacview](#).

Il vous faut un logiciel capable de visualiser ce type de fichier, par exemple "Obj Viewer" de Cem Yuksel disponible ici : <http://www.cemyuksel.com/software> (Ce logiciel n'est plus mis à jour mais fonctionne encore parfaitement).

Certains avions ou unités terrestres n'ont pas de correspondance dans Tacview. Pour ceux-ci, des correspondances se rapprochant le plus possible ont été choisies.

Par exemple : un Wellington, un Blenheim ou un Beaufighter sera représenté sous la forme d'un Mosquito, les Dornier sous forme d'un B-25, un FW 200 en B-17.

Tableau des correspondances (avions seulement).

Avion (tous variants)	Modèle 3D Tacview	Correspondance exacte
Anson	FixedWing.Mosquito.obj	N
Beaufighter	FixedWing.Mosquito.obj	N
Bf108	FixedWing.PC-21.obj	N
Bf109	FixedWing.Bf 109.obj	O
Bf110	FixedWing.Bf 110.obj	O
Blenheim	FixedWing.Mosquito.obj	N
BR20	FixedWing.Bf 110.obj	N
CR42	FixedWing.CAML.obj	N
D520	FixedWing.LaGG-3.obj	N
Defiant	FixedWing.Hurricane.obj	N
DH82	FixedWing.Dolphin.obj	N
Do17	FixedWing.B-25.obj	N
Do215	FixedWing.B-25.obj	N
FW200	FixedWing.B-17.obj	N

G50	FixedWing.I-16.obj	N
Gladiator	FixedWing.Eagle II.obj	N
He111	FixedWing.He 111.obj	O
Hurricane	FixedWing.Hurricane.obj	O
Ju87	FixedWing.Ju 87.obj	O
Ju88	FixedWing.Ju 88.obj	O
Kittyhawk	FixedWing.P-40.obj	O
C202	FixedWing.C.202.obj	O
Martlet	FixedWing.M20.obj	N
Spitfire	FixedWing.Spitfire.obj	O
Su26	FixedWing.Edge 540.obj	N
Sunderland	FixedWing.Mosquito.obj	N
Tomahawk	FixedWing.P-40.obj	O
Walrus	FixedWing.Mosquito.obj	N
Wellington	FixedWing.Mosquito.obj	N

Si un graphiste souhaite modéliser les modèles 3D manquants, il est possible d'ajouter nos propres modèles mais cela obligera de les ajouter dans un dossier particulier de Tacview. Plus d'infos sur le site : <https://www.tacview.net/documentation/3dobjects/fr/> (en anglais).

Liens utiles.

Logiciel Tacview.

<https://www.tacview.net/product/fr/>

Le WiKi de Tacview (en anglais).

https://tacview.fandom.com/wiki/Tacview_Wiki

Le GitHub de Tacview.

<https://github.com/Vyrtuoz/Tacview>

Forum ATAG.

<https://theairtacticalassaultgroup.com/forum/>

Forum ATAG, section Full Mission Builder and Scripts (creation de mission et scripts).

<https://theairtacticalassaultgroup.com/forum/forumdisplay.php?f=50>

Configuration d'un serveur multi-joueurs.

https://artist.onlyoffice.eu/Products/Files/DocEditor.aspx?fileid=6541916&doc=c0dUQ0Zta01tS0MyYjVqeJRxU2hmeloyVXNLU0s2eDZlbit0TFZYblFNUT0_ljY1NDE5MTYiO

Ressources pour missions (scripts).

<https://theairtacticalassaultgroup.com/forum/showthread.php?t=34571>

Watchdog.

<https://theairtacticalassaultgroup.com/forum/showthread.php?t=35224&highlight=watchdog>
https://www.team-fusion-simulations.de/tfsresources/?&path=03_Servers

Visionneuse de fichiers au format obj (géométrie 3D).

<http://www.cemyuksel.com/software>

Librairie .NET pour compression de fichier au format ZIP.

<https://documentation.help/DotNetZip/About.htm>

Problèmes connus.

- La balistique des projectiles (balles, obus, bombes, torpilles) n'est pas disponible et n'est pas représentée dans Tacview.
- Le recorder ne fonctionne pas pour les missions rapides et d'entraînement car ces missions sont auto-générées lors de leur lancement.
- Certains objets n'ont pas de correspondance dans la base de données des [objets 3D de Tacview](#). Le souci se présente surtout pour les objets statiques et les bâtiments et dans une moindre mesure pour les avions, les navires, les véhicules (chars, trains, ...). Pour les avions, la meilleure correspondance possible (forme, taille) disponible pour leur représentation dans Tacview a été choisie.
- La taille de l'Hurricane n'est pas correcte dans la version 1.9.3 de Tacview et antérieure. Le problème a été résolu dans la version 1.9.4 beta 4 ([nécessite un licence standard](#)).
- Léger décalage de la position géographique des objets dans Tacview. J'y travaille mais si un cartographe est disponible, je suis ouvert à toute collaboration.
- L'orientation des objets statiques est incorrecte. Cette donnée n'est pas disponible pour ce type d'objet.
- La destruction de certains bâtiments ne déclenche pas l'événement OnBuildingKilled.
- Il reste peut-être quelques bugs cachés. N'hésitez pas à les signaler sur le forum [ATAG](#).